Large Language Models Explained

A deep dive into how modern LLMs work, how they are trained, and how to use them effectively in your applications.

1. What Is a Large Language Model?

A large language model (LLM) is a neural network trained on massive text corpora to predict the next token in a sequence. Through this deceptively simple objective, models with billions of parameters learn grammar, facts, reasoning patterns, and even code. Notable examples include GPT-4, Claude, Gemini, and the open-source Llama family.

2. The Transformer Architecture

All modern LLMs are built on the transformer architecture introduced in the 2017 paper 'Attention Is All You Need'. The key components are:

Tokeniser. Converts raw text into integer tokens. Modern tokenisers (BPE, SentencePiece) balance vocabulary size against token sequence length.

Embedding layer. Maps each token ID to a dense vector of typically 512–8192 dimensions.

Positional encoding. Adds information about each token's position in the sequence, since transformers have no built-in notion of order.

Multi-head self-attention. Allows every token to attend to every other token in the context. Multiple heads learn different relationship types in parallel.

Feed-forward network. A two-layer MLP applied independently to each token position, adding capacity for factual recall and reasoning.

Layer normalisation. Stabilises training by normalising activations before or after each sub-layer.

Output projection. Maps the final hidden states back to vocabulary logits, from which the next token is sampled.

3. Training an LLM

Training a frontier LLM happens in three phases:

Pre-training. The model is trained on trillions of tokens from the web, books, and code using next-token prediction. This costs millions of dollars in compute.

Supervised Fine-Tuning (SFT). Human-written demonstrations of helpful, harmless responses are used to fine-tune the base model into a chat-capable assistant.

Reinforcement Learning from Human Feedback (RLHF). Human raters rank model outputs; a reward model is trained on these rankings; the LLM is then optimised against the reward model using PPO or a simpler algorithm like DPO.

4. Context Windows and Memory

An LLM's context window is the maximum number of tokens it can attend to at once. Early GPT models had 2 048 tokens; modern models support 128 000 or more. Despite large context windows, LLMs have no persistent memory between conversations — every session starts fresh. Techniques to add memory include vector databases (RAG), external key-value stores, and fine-tuning on user-specific data.

5. Prompting Techniques

Zero-shot prompting. Ask the model directly without examples. Works for tasks within its training distribution.

Few-shot prompting. Provide 2–5 input-output examples before your query. Dramatically improves consistency on structured tasks.

Chain-of-thought (CoT). Instruct the model to 'think step by step'. Unlocks multi-step reasoning on maths and logic problems.

ReAct. Combines reasoning and tool-calling in an interleaved loop: Thought -> Action -> Observation -> Thought.

System prompts. Set the model's persona, constraints, and output format before the conversation begins.

6. Retrieval-Augmented Generation (RAG)

RAG is a technique that augments an LLM with a retrieval step. When the user asks a question, relevant passages are fetched from a vector database and injected into the prompt as context. This allows the model to answer questions about documents it was never trained on, dramatically reduces hallucination, and provides citable sources. RAG is the preferred architecture for enterprise document Q&A; systems.

7. Common LLM Failure Modes

Hallucination. The model generates plausible-sounding but factually incorrect information. Mitigate with RAG and grounding.

Sycophancy. The model agrees with the user even when they are wrong. Mitigate with RLHF and explicit instructions to disagree.

Prompt injection. Malicious text in retrieved documents tries to hijack the model's instructions. Mitigate with input sanitisation.

Context dilution. Important information gets 'lost in the middle' of long contexts. Mitigate with chunking and re-ranking.

Verbose refusals. Over-cautious safety training causes the model to refuse benign requests. Mitigate with better system prompts.